Debugging using the GNU Debugger gdb

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026

Using gdb in Assignments, How To start Debugging xv6 kernel (1 of 3)

- Build and start a gdb server running xv6 (in first window)
  - CPUS=1 will restrict to a single hartid (simulated RISC-V hardware thread)
  - ctrl-a x to exit the qemu emulator running the server window

```
student@xv6os:xv6-riscv$ make CPUS=1 qemu-gdb
riscv64-linux-gnu-gcc -march=rv64gc -g -c -o kernel/entry.o kernel/entry.S
...
balloc: write bitmap block at sector 46
*** Now run 'gdb' in another window.
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel
    -m 128M -smp 1 -nographic -global virtio-mmio.force-legacy=false -drive
    file=fs.img,if=none,format=raw,id=x0
    -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0 -S -gdb tcp::26000
```

- This creates a file in current directory named `.gdbinit`

EAST TEXAS A&M

Using gdb in Assignments, How To start Debugging xv6 kernel (2 of 3)

- Open a second terminal (leave server still running in first terminal)
- Confirm your `.gdbinit` file, should look like this:
  - port number is port the xv6 debug server is listening on

```
student@xv6os:xv6-riscv$ cat .gdbinit
set confirm off
set architecture riscv:rv64
target remote 127.0.0.1:26000
symbol-file kernel/kernel
set disassemble-next-line auto
set riscv use-compressed-breakpoints yes
```

- First time you run, need to allow gdb to use settings from this file
  - create `~/.config/gdb/gdbinit` file with "set auto-load safe-path /"

```
student@xv6os:xv6-riscv$ mkdir -p ~/.config/gdb
student@xv6os:xv6-riscv$ echo "set auto-load safe-path /" >> ~/.config/gdb/gdbinit
student@xv6os:xv6-riscv$ cat ~/.config/gdb/gdbinit
set auto-load safe-path /
```

EAST TEXAS A&M

Using gdb in Assignments, How To start Debugging xv6 kernel (3 of 3)

- Still with the server running, and once you have `gdbinit` settings set, run `gdb` client
  - The `-tui` will start in text user interface layout, more on this later
  - set breakpoint in syscall
  - continue executing, will break in syscall
  - print out info about the current process

```
student@xv6os:xv6-riscv$ gdb-multiarch -tui
...
(gdb) b syscall
Breakpoint 1 at 0x800027e4: file kernel/syscall.c, line 138.
(gdb) c
Continuing.

Breakpoint 1, syscall () at kernel/syscall.c:138
```

EAST TEXAS A&M

Using gdb in Assignments, Text User Interface

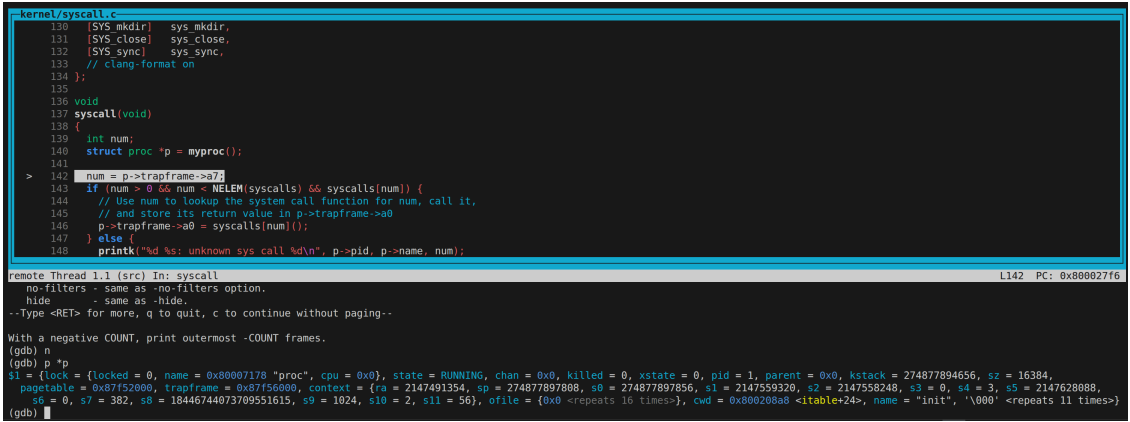

Figure 1: Example of gdb running with text user interface layout src

EAST TEXAS A&M

GDB Commands

- See Additional Resources, GDB Cheat Sheet (Haisenko, 2007)
- Run `help <command-name>` if you're not sure how to us a command
- All commands may be abbreviated if unambiguous
  - `c` = `co` = `cont` = `continue`
- Some additional abbreviations are defined for often used commands, e.g.
  - `s` = `step` and `si` = `stepi`

EAST TEXAS A&M

Stepping

Stepping through code, a line at a time, is a basic thing you might want to do. Can step by C statements, or by assembly instructions.

- `step` or `s` runs one line of code at a time
  - `step` steps **into** function calls
- `next` or `n` does same thing, except that it steps **over** function calls
- `stepi` and `nexti` do the same thing for assembly instructions rather than lines of (C) code
- All take a numerical argument to specify repetition
- Pressing the enter key repeats the previous command

EAST TEXAS A&M

Running

- `continue` runs code until a breakpoint is encountered or you interrupt it with ctrl-c
- `finish` runs code until the current function returns
- `advance <location>` runs code until the instruction pointer (PC) gets to the specified location

EAST TEXAS A&M

Breakpoints

A common task in debugging is to want the debugger to stop executing at some point in the code so you can begin stepping and examining

- `break <location>` sets a breakpoint at the specified location
- Locations can be memory addresses (`*0x7c00`) or names (`mon_backtrace`, `syscall`, `monitor.c:71`)
- Modify breakpoints using `delete`, `disable`, `enable`
- List breakpoints `info break`

EAST TEXAS A&M

Conditional Breakpoints

- `break <location> if <condition>`
  - sets breakpoint at specified location, but it only breaks if the condition is satisfied
- `cond <number> <condition>` adds a condition on an existing breakpoint
  - Using `info break` mentioned to get break point numbers

EAST TEXAS A&M

Watchpoints

Like breakpoints, but with more complicated conditions

- `watch <expression>` will stop execution whenever the expression's value changes
- `watch -l <address>` will stop execution whenever the contents of the contents of the specified memory address change

What's the difference between `wa var` and `wa -l &var`?

- `rwatch [-l] <expression>` will stop execution whenever the value of the expression is read

EAST TEXAS A&M

Examining

- `x` prints the raw contents of memory in whatever format you specify
  - `x/x` for hexadecimal
  - `x/i` for assembly instruction
  - remember `help x` to see full list of formats for examine
- `print` evaluates a C expression and prints the result as its proper type
  - Often more useful than `x` when examining C structures
  - The output from `p *((struct elfhdr *) 0x10000)` is much nicer than raw hex output from `x/13x 0x10000`

More Examining

- `info registers` prints the value of every register
- `info frame` prints the current stack frame
- `list <location>` prints the source code of the function at the specified location
- `backtrace` gives trace of all stack frames

Layouts

- gdb has a text user interface (tui) that show useful information in curses UI
  - code listing
  - disassembly
  - register contents
- `layout <name>` switches to the given layout

EAST TEXAS A&M

Other Tricks

- You can use the `set` command to actually change the value of a variable while executing
- You have to switch symbol files to get function and variable names for environments other than kernel
  - kernel symbols are loaded by the `.gdbinit` command `symbol-file kernel/kernel`
  - When debugging a user program may need to load its symbols `symbol-file user/_find`

Summary

- Read the manual! use the `help` command
- `gdb` is tremendously powerful and this only scratches the surface of its features
- It is well worth your time (for this class, for your career) learning more of how to use `gdb` specifically and in general using proper debugging tools

EAST TEXAS A&M

References

Haisenko, M. (2007). *GDB cheat sheet*. Retrieved August 1, 2026, from https://darkdust.net/files/GDB%20Cheat%20Sheet.pdf

EAST TEXAS A&M